

## ITA0608 - Machine Learning

### List of Lab Exercises

#### Experiment 9: Compare Linear and Polynomial Regression using Python

##### Aim

To compare the performance of Linear Regression and Polynomial Regression models using Python.

##### Software Required

- Python 3.x
- NumPy
- Matplotlib
- Scikit-learn

##### Theory

**Linear Regression** models the relationship between input and output using a straight line.

$$\hat{y} = b_0 + b_1x$$

$$\hat{y} = 8.49 - 0.54x$$

$R^2 = 0.72$  ·  $b_0$  = intercept ·  $b_1$  = slope · least squares minimizes squared vertical residual gaps.

Focus

DataLinePredictionsResiduals

DataLinePredictionsResiduals

Drag a point to refit the line.

246810246810

**Polynomial Regression** extends Linear Regression by fitting a curved relationship using polynomial features.

A polynomial model can be represented as:

$$y = b_0 + b_1x + b_2x^2 + \dots + b_nx^n$$

##### Algorithm

1. Import required libraries.
2. Create the dataset.

3. Apply Linear Regression and train the model.
4. Generate polynomial features.
5. Apply Polynomial Regression and train the model.
6. Predict outputs using both models.
7. Plot and compare the results.

### **Program**

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

# Dataset

X = np.array([1, 2, 3, 4, 5, 6, 7]).reshape(-1, 1)
Y = np.array([1, 4, 9, 16, 25, 36, 49])

# Linear Regression

lin_reg = LinearRegression()
lin_reg.fit(X, Y)

# Polynomial Regression

poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)
poly_reg = LinearRegression()
poly_reg.fit(X_poly, Y)

# Plot

plt.scatter(X, Y, color='blue', label='Data')

plt.plot(X, lin_reg.predict(X),
         color='red', label='Linear Regression')

plt.plot(X, poly_reg.predict(X_poly),
         color='green', label='Polynomial Regression')
```

```
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Linear vs Polynomial Regression')
plt.legend()
plt.show()
```

## # Predictions

```
print("Linear Regression Predictions:")
print(lin_reg.predict(X))
```

```
print("\nPolynomial Regression Predictions:")
print(poly_reg.predict(X_poly))
```

## Output

```
[2]
✓ Os  import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

# Dataset
X = np.array([1, 2, 3, 4, 5, 6, 7]).reshape(-1, 1)
Y = np.array([1, 4, 9, 16, 25, 36, 49])

# Linear Regression
lin_reg = LinearRegression()
lin_reg.fit(X, Y)

# Polynomial Regression
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

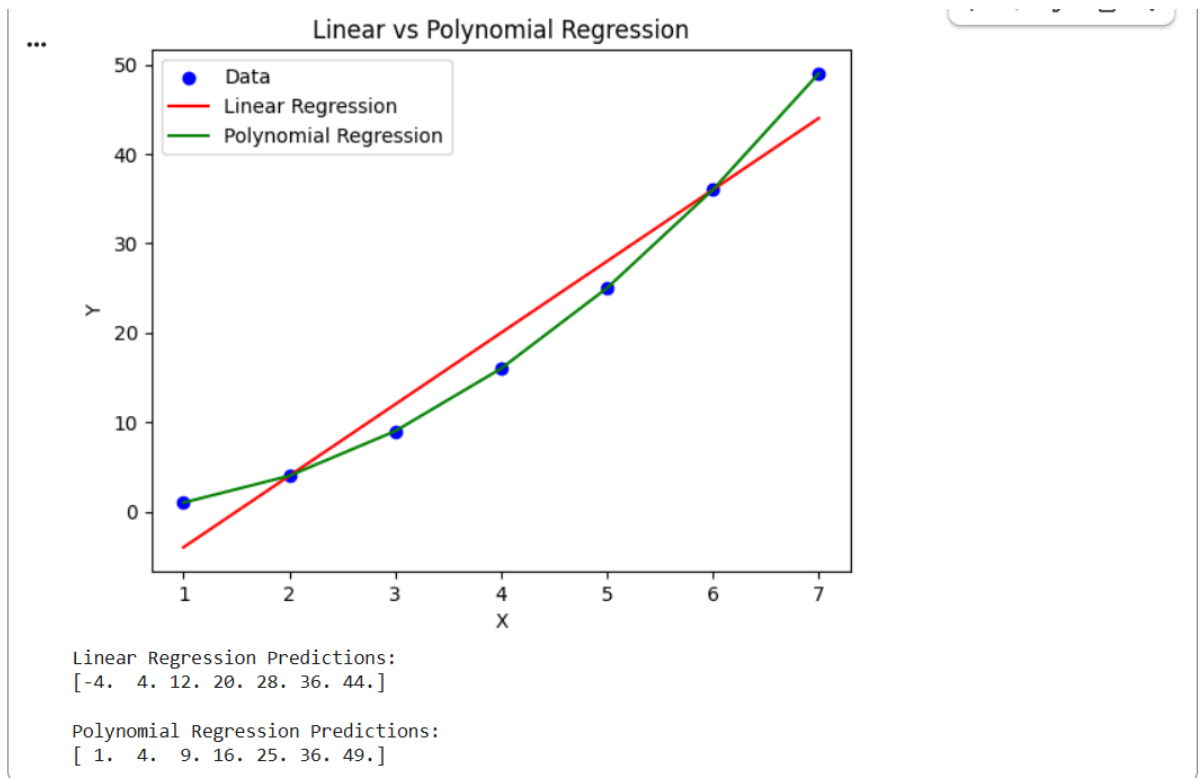
poly_reg = LinearRegression()
poly_reg.fit(X_poly, Y)

# Plot
plt.scatter(X, Y, color='blue', label='Data')

plt.plot(X, lin_reg.predict(X),
         color='red', label='Linear Regression')

plt.plot(X, poly_reg.predict(X_poly),
         color='green', label='Polynomial Regression')

plt.xlabel('X')
plt.ylabel('Y')
```



- A graph displaying:
  - **Red Line** → Linear Regression
  - **Green Curve** → Polynomial Regression
- Polynomial Regression fits the curved data better than Linear Regression.

Accuracy Lower for curved data Higher for curved data

## Result

Thus, Linear Regression and Polynomial Regression were implemented and compared. Polynomial Regression provided a better fit for non-linear data, while Linear Regression was suitable only for linear relationships.